

EE451 PHW3

Rafael Wang

rafaelwa@usc.edu

USC ID: 6189106477

For the entire PHW, the test platform used was the USC CARC Discovery cluster. The allocated compute node had the configuration `--cpus-per-task=32 --mem=8G`. The number of runs performed for each problem was 1. In terms of other necessary variables, the compiler used was `g++`.

For `p1a`, `p1b`, and `p1c`, the matrix size is $2K \times 2K$.

For `p1a`, the execution time for $p \times p = 1$ is shown in the output below:

```
[bash-4.4$ ./p1a 1  
Execution time = 101.107 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 4$ is shown in the output below:

```
[bash-4.4$ ./p1a 2  
Execution time = 26.1964 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 16$ is shown in the output below:

```
[bash-4.4$ ./p1a 4  
Execution time = 8.57491 sec,  
C[100][100]=2.30093e+08
```

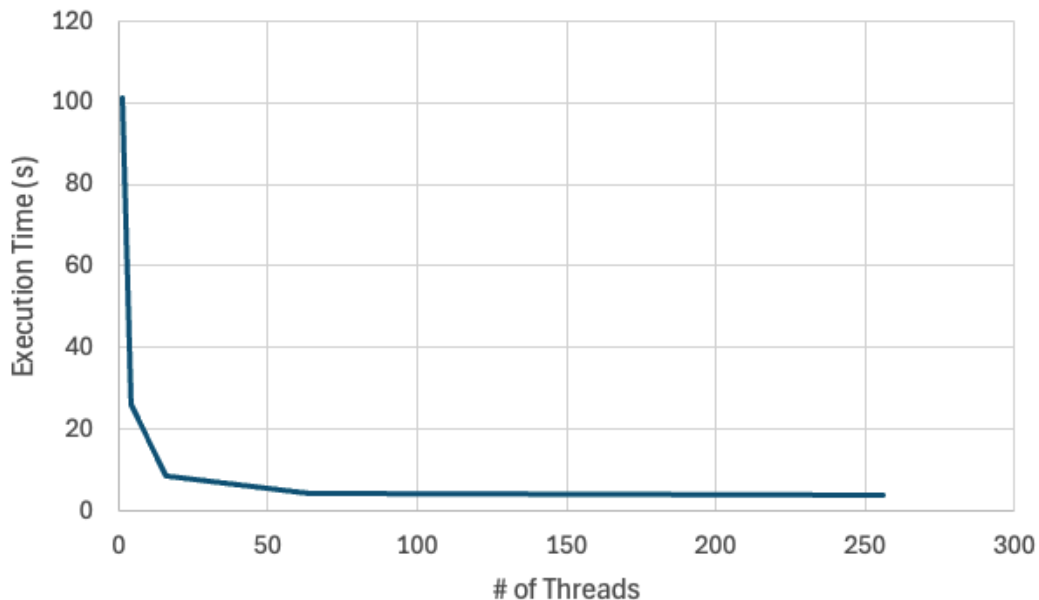
The execution time for $p \times p = 64$ is shown in the output below:

```
[bash-4.4$ ./p1a 8  
Execution time = 4.2012 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 256$ is shown in the output below:

```
[bash-4.4$ ./p1a 16  
Execution time = 3.7094 sec,  
C[100][100]=2.30093e+08
```

The diagram showing execution times under various p is shown below:



For $p1b$, the execution time for $p \times p = 1$ is shown in the output below:

```
[bash-4.4$ ./p1b 1  
Execution time = 98.7689 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 4$ is shown in the output below:

```
[bash-4.4$ ./p1b 2  
Execution time = 24.728 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 16$ is shown in the output below:

```
[bash-4.4$ ./p1b 4  
Execution time = 6.48238 sec,  
C[100][100]=2.30093e+08
```

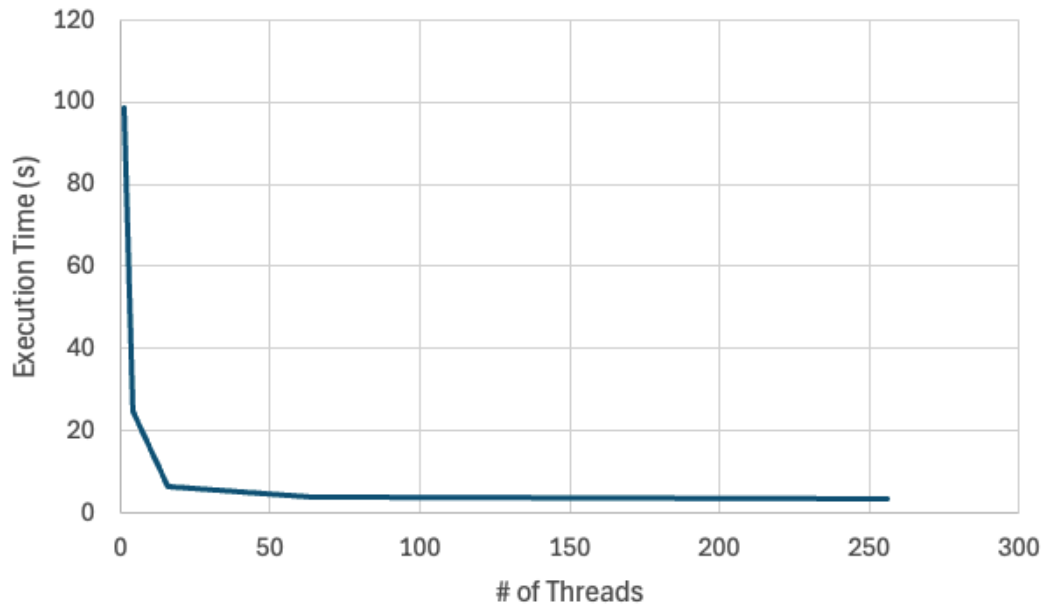
The execution time for $p \times p = 64$ is shown in the output below:

```
[bash-4.4$ ./p1b 8  
Execution time = 3.73468 sec,  
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 256$ is shown in the output below:

```
[bash-4.4$ ./p1b 16
Execution time = 3.40571 sec,
C[100][100]=2.30093e+08
```

The diagram showing execution times under various p is shown below:



For p1c, the execution time for $p \times p = 1$ is shown in the output below:

```
[bash-4.4$ ./p1c 1
Execution time = 97.2346 sec,
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 4$ is shown in the output below:

```
[bash-4.4$ ./p1c 2
Execution time = 24.3235 sec,
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 16$ is shown in the output below:

```
[bash-4.4$ ./p1c 4
Execution time = 6.22112 sec,
C[100][100]=2.30093e+08
```

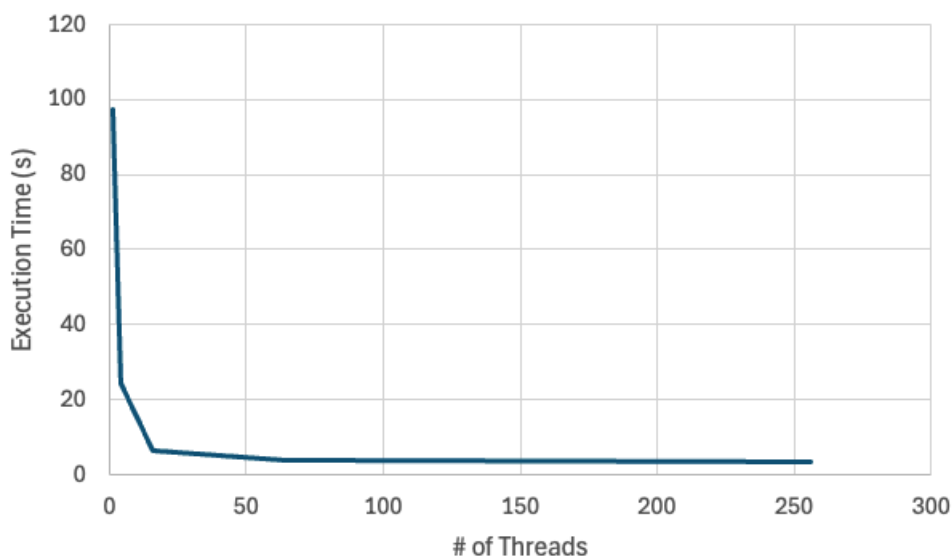
The execution time for $p \times p = 64$ is shown in the output below:

```
[bash-4.4$ ./p1c 8
Execution time = 3.58584 sec,
C[100][100]=2.30093e+08
```

The execution time for $p \times p = 256$ is shown in the output below:

```
[bash-4.4$ ./p1c 16
Execution time = 3.39008 sec,
C[100][100]=2.30093e+08
```

The diagram showing execution times under various p is shown below:



In terms of my observations, I notice that Version C consistently outperforms Version B, and version B consistently outperforms version A at each number of threads. In all three versions, execution time drastically improves as threads are increased, and it begins to plateau after around 16 threads. The performance differs between the three versions as Version A makes poor use of cache locality since matrix B is accessed in column-major fashion, but multi-dimensional arrays in C++ are stored in row-major format, causing it to have the worst performance. Cache locality is improved in Version B as blocked matrix multiplication allows values to be reused while they are still in the cache. Version C performs best because it transposes matrix B, which means it is stored in column-major format. Since we are now accessing matrix B_Transpose in row-major format (i.e., B in column-major format), this improves cache locality, ultimately leading to the best performance.

The execution time for the serial version of Estimating pi is shown in the output below:

```
[bash-4.4$ ./p2_serial
Estimated pi is 3.142402, execution time = 0.235985 sec
```

For p2a, the execution time is shown in the output below:

```
[bash-4.4$ ./p2a  
Estimated pi is 3.142402, execution time = 0.058221 sec
```

For p2b, the execution time is shown in the output below:

```
[bash-4.4$ ./p2b  
Estimated pi is 3.142402, execution time = 0.118746 sec
```

The execution time of the serial version of Quicksort is shown in the output below:

```
[bash-4.4$ ./p3_serial  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16  
Execution time = 0.239028 sec
```

For p3, the execution time is shown in the output below:

```
[bash-4.4$ ./p3  
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16  
Execution time = 0.202570 sec
```

After changing the initialization to $m[i] = \text{rand}()$, the execution time is shown in the output below:

```
126 1888 2429 2801 3557 4880 5148 7006 7483 8731 9625 12044 12913 15622 15708 16011  
Execution time = 0.286788 sec
```

There is a performance difference in which using $m[i]=\text{size}-i$ as the initialization produces a better execution time than using $m[i]=\text{rand}()$. It is typically expected that random initialization would produce better execution time since its random nature makes it more likely that the partitioning would be more even. Additionally, initializing as $m[i]=\text{size}-i$ would cause very imbalanced partitioning, particularly if the start or end element is chosen as the pivot. However, since a random pivot was used in my implementation of Quicksort, the partitioning was likely not as imbalanced, which is one possible reason why the version using $m[i]=\text{size}-i$ actually outperformed using $m[i]=\text{rand}()$.